

```

require('dotenv').config();
const express = require('express');
const multer = require('multer');
const cors = require('cors');
const path = require('path');
const fs = require('fs-extra');
const { v4: uuidv4 } = require('uuid');
const Anthropic = require('@anthropic-ai/sdk');

const app = express();
const PORT = process.env.PORT || 3000;

const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

app.use(cors());
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ extended: true, limit: '50mb' }));
app.use(express.static(path.join(__dirname, 'public')));
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

fs.ensureDirSync(path.join(__dirname, 'uploads'));
fs.ensureDirSync(path.join(__dirname, 'proposals'));
fs.ensureDirSync(path.join(__dirname, 'tracking'));

// Multer - handle both images and PDFs
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const proposalId = req.body.proposalId || 'temp';
    const dir = path.join(__dirname, 'uploads', proposalId);
    fs.ensureDirSync(dir);
    cb(null, dir);
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, `${Date.now()}-${uuidv4().slice(0,8)}${ext}`);
  }
});

const upload = multer({ storage, limits: { fileSize: 30 * 1024 * 1024 } });

// — ROUTES —

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

```

```

// Upload images
app.post('/api/upload', upload.array('images', 20), (req, res) => {
  try {
    const files = req.files.map(f => ({
      filename: f.filename,
      originalname: f.originalname,
      url: `/uploads/${req.body.proposalId}/${f.filename}`,
      category: req.body.category || 'general'
    }));
    res.json({ success: true, files });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Upload PDF estimate - reads it and returns structured data via Claude
app.post('/api/parse-estimate', upload.single('pdf'), async (req, res) => {
  try {
    const filePath = req.file.path;
    const pdfBuffer = await fs.readFile(filePath);
    const base64Pdf = pdfBuffer.toString('base64');

    const message = await anthropic.messages.create({
      model: 'claude-opus-4-5',
      max_tokens: 2000,
      messages: [{
        role: 'user',
        content: [
          {
            type: 'document',
            source: { type: 'base64', media_type: 'application/pdf', data: base64
          },
          {
            type: 'text',
            text: `You are reading an internal renovation estimate (CONFIDENTIAL)

```

The JSON must have this exact shape:

```

{
  "clientName": "string",
  "projectAddress": "string",
  "projectTitle": "string - short description of the main work (e.g. Kitchen Cabi
  "duration": "string - e.g. 10 days",
  "grandTotal": number,
  "scopeCategories": [
    {
      "title": "string - clean client-facing category name (e.g. Countertop Repla

```

```

      "description": "string - 1-2 sentences describing what this covers in proje
      "icon": "string - one of: countertop, cabinet, paint, plumbing, flooring, e
    }
  ],
  "lineItems": [
    { "label": "string - clean client-facing label", "amount": number }
  ],
  "notes": "string - any relevant project notes"
}

```

#### IMPORTANT:

- scopeCategories should group related work (do NOT list every line item - group
- lineItems should show ONLY the client-facing budget categories (e.g. "Cabinetry
- grandTotal is the final total the client will pay
- Never expose internal cost breakdowns, markup percentages, or subcontractor cos

```

    }
  ]
}
});

const raw = message.content[0].text.replace(/```json|```/g, '').trim();
const parsed = JSON.parse(raw);

// Clean up temp file
await fs.remove(filePath);

res.json({ success: true, data: parsed });
} catch (err) {
  console.error('PDF parse error:', err);
  res.status(500).json({ error: err.message });
}
});

```

```

// Generate full proposal HTML
app.post('/api/generate', async (req, res) => {
  const { proposalData, images } = req.body;
  try {
    const prompt = buildPrompt(proposalData, images);
    const message = await anthropic.messages.create({
      model: 'claude-opus-4-5',
      max_tokens: 8000,
      messages: [{ role: 'user', content: prompt }]
    });

    const htmlContent = message.content[0].text;
    const proposalId = proposalData.proposalId || uuidv4();
    const filename = `proposal-${proposalId}.html`;

```

```

const filepath = path.join(__dirname, 'proposals', filename);

const trackedHtml = injectTracking(htmlContent, proposalId);
await fs.writeFile(filepath, trackedHtml);

await fs.writeJson(path.join(__dirname, 'tracking', `${proposalId}.json`), {
  proposalId,
  clientName: proposalData.clientName,
  projectAddress: proposalData.projectAddress,
  totalAmount: proposalData.grandTotal || proposalData.totalAmount,
  createdAt: new Date().toISOString(),
  views: [],
  totalTimeSeconds: 0,
  lastViewed: null
});

res.json({ success: true, proposalId, viewUrl: `/view/${proposalId}` });
} catch (err) {
  console.error(err);
  res.status(500).json({ error: err.message });
}
});

app.get('/view/:proposalId', (req, res) => {
  const filepath = path.join(__dirname, 'proposals', `proposal-${req.params.propo
  if (fs.existsSync(filepath)) {
    res.sendFile(filepath);
  } else {
    res.status(404).send('Proposal not found');
  }
});

app.post('/api/track', async (req, res) => {
  const { proposalId, event, duration } = req.body;
  const trackFile = path.join(__dirname, 'tracking', `${proposalId}.json`);
  if (!fs.existsSync(trackFile)) return res.json({ ok: true });
  try {
    const data = await fs.readJson(trackFile);
    if (event === 'open') {
      data.views.push({
        openedAt: new Date().toISOString(),
        userAgent: req.body.userAgent || '',
        referrer: req.body.referrer || '',
        duration: 0,
        sessionId: req.body.sessionId
      });
    }
    data.lastViewed = new Date().toISOString();
  }
});

```

```

    }
    if (event === 'close' || event === 'heartbeat') {
      const session = data.views.find(v => v.sessionId === req.body.sessionId);
      if (session) session.duration = duration || 0;
      data.totalTimeSeconds = data.views.reduce((sum, v) => sum + (v.duration || 0));
    }
    await fs.writeJson(trackFile, data);
    res.json({ ok: true });
  } catch (err) {
    res.json({ ok: true });
  }
});

app.get('/api/proposals', async (req, res) => {
  try {
    const trackingDir = path.join(__dirname, 'tracking');
    const files = await fs.readdir(trackingDir);
    const proposals = await Promise.all(
      files.filter(f => f.endsWith('.json')).map(f => fs.readJson(path.join(trackingDir, f)))
    );
    proposals.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
    res.json(proposals);
  } catch (err) {
    res.json([]);
  }
});

app.get('/api/proposals/:id', async (req, res) => {
  try {
    const data = await fs.readJson(path.join(__dirname, 'tracking', `${req.params.id}.json`));
    res.json(data);
  } catch (err) {
    res.status(404).json({ error: 'Not found' });
  }
});

app.use('/proposals', express.static(path.join(__dirname, 'proposals')));

// — PROMPT BUILDER —————

function buildPrompt(data, images) {
  const { clientName, projectAddress, scopeCategories, lineItems, grandTotal,
    companyTagline, notes, projectTitle, duration } = data;

  const beforeAfterImages = images?.filter(i => i.category === 'before_after') || []
  const renderImages = images?.filter(i => i.category === 'render') || []
  const projectImages = images?.filter(i => i.category === 'project') || []

```

```

    const imagesSection = images?.length ? `
  IMAGES (embed these src URLs directly):
  Before/After: ${JSON.stringify(beforeAfterImages.map(i => i.url))}
  3D Renders: ${JSON.stringify(renderImages.map(i => i.url))}
  Hero/Project Photos: ${JSON.stringify(projectImages.map(i => i.url))}
  ` : '';

    const scopeHtml = scopeCategories?.map(s => `- ${s.title}: ${s.description}`).join('')
    const lineItemsHtml = lineItems?.map(l => `${l.label}: ${Number(l.amount).toLocaleString()}`)

    return `You are a world-class renovation proposal designer. Create a stunning,

  BRAND:
  - Company: Formo Renovation LLC
  - Palette: #191816 background, #C9A96E gold accent, #F8F4EF cream text, #cfc7b8 m
  - Fonts: Google Fonts - "Cormorant Garamond" headings + "Jost" body
  - Tagline: "${companyTagline || 'Built on quality, driven by integrity'}"

  PROJECT:
  - Client: ${clientName}
  - Project: ${projectTitle || 'Renovation Project'}
  - Address: ${projectAddress || ''}
  - Duration: ${duration || ''}
  - Notes: ${notes || ''}

  SCOPE CATEGORIES (use these for the services section - no prices per category):
  ${scopeHtml}

  INVESTMENT (these are the only numbers the client sees):
  ${lineItemsHtml}
  TOTAL: ${Number(grandTotal).toLocaleString()}

  ${imagesSection}

  SECTIONS TO INCLUDE:
  1. HERO - Full viewport. Client name prominent. Project title. Dark, dramatic. If
  2. THE WORK - One card per scope category. Each card has: icon (inline SVG), cate
  3. ${beforeAfterImages.length > 0 ? 'BEFORE & AFTER - Split or slider comparison'
  4. ${renderImages.length > 0 ? 'RENDERINGS - Full-width render showcase' : ''}
  5. INVESTMENT - Premium budget display. Show line items grouped (e.g. "Cabinetry
  6. WHY FORMO - 3 pillars: Licensed & Insured · On-Time Delivery · Premium Craftsm
  7. NEXT STEPS - Dan Lares · dan@formorenovation.com · 615-608-1220 · formorenovat

  DESIGN RULES:
  - Single self-contained HTML file, all CSS in <style> tag
  - Google Fonts import at top

```

- Cormorant Garamond for all headings, Jost for body
- Scroll animations: use IntersectionObserver to add a class like "visible" that  
 CSS: `.fade { opacity:0; transform:translateY(24px); transition:opacity .6s ease`  
`.fade.visible { opacity:1; transform:translateY(0); }`  
 JS: `const obs = new IntersectionObserver(entries => entries.forEach(e => { if(e`  
`document.querySelectorAll('.fade').forEach(el => obs.observe(el));`  
 ALSO add this fallback after the observer: `setTimeout(() => document.querySelec`
- Gold diamond (◆) dividers between sections
- Mobile responsive
- Luxury feel – NOT a contractor quote sheet
- The investment section total should be displayed at ~64px in gold

OUTPUT: Return ONLY the HTML. Start with `<!DOCTYPE html>`. No markdown. No explana  
 }

```
// — TRACKING INJECTION —————

function injectTracking(html, proposalId) {
  const script = `
<script>
(function() {
  var PID = '${proposalId}';
  var SID = Math.random().toString(36).slice(2);
  var BASE = window.location.origin;
  var t0 = Date.now();
  var on = true;
  function ping(ev) {
    var body = JSON.stringify({ proposalId:PID, sessionId:SID, event:ev,
      duration:Math.round((Date.now()-t0)/1000), userAgent:navigator.userAgent, r
    try { navigator.sendBeacon(BASE+'/api/track', new Blob([body],{type:'applicat
    }
    ping('open');
    setInterval(function(){ if(on) ping('heartbeat'); }, 30000);
    window.addEventListener('beforeunload', function(){ ping('close'); });
    document.addEventListener('visibilitychange', function(){
      if(document.hidden){ ping('close'); on=false; } else { t0=Date.now(); on=true
    });
  })();
</script>`;
  return html.replace('</body>', script + '\n</body>');
}

app.listen(PORT, () => {
  console.log(`Formo Proposal Generator running on port ${PORT}`);
});
```

